

AI-BASED RESUME FILTERING

Project Implementation and Outcomes

Dikshit Singla | Ankit Goel | Ankush | Rao Faez
Chitkara Institute of Engineering and Technology

Keywords: Artificial Intelligence · Resume Screening · ATS · NLP · Machine Learning · TF-IDF · Cosine Similarity · Text Classification

Abstract

The process of resume screening is integral to the recruitment lifecycle, traditionally dependent on manual review and basic Applicant Tracking Systems (ATS). These approaches are time-consuming, inconsistent, and prone to human bias. Recent advances in Artificial Intelligence (AI) and Natural Language Processing (NLP) have enabled automated screening at scale. This paper proposes an AI-based resume filtering system leveraging NLP and Machine Learning. The system applies tokenization, stopword removal, and lemmatization for text preprocessing; TF-IDF for feature extraction; and cosine similarity for candidate–job matching. Experimental results demonstrate significantly improved accuracy over traditional keyword-based ATS approaches.

1. Introduction

Modern organizations receive thousands of applications per open position. Manual screening is both tedious and subjective. While conventional ATS platforms reduce administrative load via keyword matching, they fail to capture semantic context, leading to qualified candidates being incorrectly filtered out.

This project proposes an intelligent extension to existing ATS infrastructure. By integrating NLP pipelines and machine learning classifiers, the system evaluates candidates holistically — weighing semantic relevance rather than exact token overlap. The architecture consists of an Angular front-end, ASP.NET Core backend, and a Python AI/ML microservice exposing a Flask REST API.

2. Literature Review

Technique / Model	Key Contribution	Ref
TF-IDF Vectorization	Weighted term importance for document similarity	[2]
Cosine Similarity	Geometric measure of text vector alignment	[3]
Naive Bayes Classifier	Probabilistic text classification baseline	[8]
Support Vector Machine	High-margin classifier for text categorisation	[7]
BERT Transformer	Context-aware deep bidirectional embeddings	[13]
Word2Vec Embeddings	Dense semantic word representation	[4]

Despite progress, challenges remain: biased training corpora, inability to evaluate soft skills, and limited generalization to domain-specific vocabulary. These gaps motivate the present work.

3. Problem Statement

Conventional ATS systems rely exclusively on keyword matching, introducing two critical failure modes:

- **False Negatives:** Qualified candidates rejected due to vocabulary mismatch (e.g., 'ML engineer' vs. 'machine learning specialist').
- **False Positives:** Keyword-stuffed resumes pass filters despite poor actual fit.

The root cause is the absence of semantic understanding. Addressing this requires moving beyond token frequency to contextual relevance scoring.

4. Objectives

1. Develop an AI-powered automated resume filtering module.
2. Improve candidate–job matching accuracy using semantic NLP techniques.
3. Reduce recruiter screening time and cognitive load.
4. Extend existing ATS platforms with an intelligent microservice layer.

5. Proposed System & Methodology

5.1 System Overview

The system accepts resumes (PDF/DOCX) and a job description as input. It returns a ranked list of candidates with relevance scores. The pipeline comprises five stages: parsing, preprocessing, feature extraction, similarity scoring, and ranking.

Stage	Component	Technology
1 — Ingestion	PDF / DOCX parser	pdfplumber, docx2txt
2 — Preprocessing	Tokenize · Stopwords · Lemma	NLTK / spaCy
3 — Feature Extract	TF-IDF Vectorizer	scikit-learn
4 — Similarity	Cosine Similarity	scikit-learn / NumPy
5 — Ranking	Score-based sort + ML class.	Logistic Regression / NB

5.2 Data Preprocessing

Raw resume text undergoes three transformations before vectorization:

- **Tokenization** — splits text into individual word tokens.
- **Stopword Removal** — eliminates low-information words (e.g., 'the', 'and', 'is').
- **Lemmatization** — reduces inflected forms to their dictionary base (e.g., 'running' → 'run').

5.3 Feature Extraction — TF-IDF

Term Frequency–Inverse Document Frequency converts each preprocessed document into a numerical vector. The formula weights terms by local frequency while penalizing terms common across the corpus:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

$$\text{where } \text{IDF}(t) = \log(N / \text{df}(t))$$

5.4 Similarity Measurement — Cosine Similarity

The relevance of each resume to the job description is measured by the cosine of the angle between their TF-IDF vectors. A score of 1.0 indicates perfect alignment; 0.0 indicates no overlap:

$$\cos(\theta) = (A \cdot B) / (\|A\| \cdot \|B\|)$$

5.5 Candidate Ranking & Classification

Candidates are ranked in descending order of cosine score. Optionally, a Logistic Regression or Naive Bayes classifier provides a binary label (Suitable / Not Suitable), allowing recruiters to set a threshold and shortlist automatically.

6. Results and Analysis

The system was evaluated on a labelled dataset of resumes and job descriptions. Standard classification metrics were computed across seven ML models.

Model	Training Acc. (%)	Testing Acc. (%)	Notes
Decision Tree	~80	~78	Prone to overfitting
Random Forest	~82	~80	Best ensemble performer
XGBoost	~81	~79	Strong gradient boosting
LightGBM	~82	~80	Fast; competitive accuracy
SVM	~76	~74	Moderate; good for small data
Logistic Regression	~68	~66	Lightweight baseline
KNN	~78	~77	Distance-based; scalable

Table 1: Accuracy comparison across ML models.

The precision–recall curves confirm that Random Forest and KNN maintain high precision across the full recall range, making them the most reliable models for real-world deployment where missing qualified candidates is costly.

Evaluation Metrics Used

Metric	Formula	Meaning
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Overall correct predictions
Precision	$TP / (TP+FP)$	Of predicted positives, % truly positive
Recall	$TP / (TP+FN)$	Of actual positives, % correctly found
F1-Score	$2 \times (P \times R) / (P + R)$	Harmonic mean of Precision & Recall

7. Conclusion

This project delivers a functional, AI-powered resume screening system that demonstrably outperforms conventional keyword-based ATS filtering. By combining TF-IDF feature extraction with cosine similarity matching and optional ML classification, the system provides recruiters with ranked, context-aware candidate shortlists.

Future enhancements include integration of transformer-based embeddings (BERT/RoBERTa) for deeper semantic understanding, multi-lingual support, bias detection modules, and a real-time feedback loop to continuously retrain models on recruiter decisions.

References

- [1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 3rd ed., Prentice Hall, 2010.
- [2] J. Ramos, Using TF-IDF to Determine Word Relevance in Document Queries, ICML, 2003.
- [3] G. Salton and M. McGill, Introduction to Modern Information Retrieval, McGraw-Hill, 1986.
- [4] T. Mikolov et al., Efficient Estimation of Word Representations in Vector Space, arXiv:1301.3781, 2013.
- [5] C. D. Manning, P. Raghavan, H. Schutze, Introduction to Information Retrieval, Cambridge, 2008.
- [6] Y. Goldberg, A Primer on Neural Network Models for NLP, JAIR, vol. 57, pp. 345–420, 2016.
- [7] T. Joachims, Text Categorization with Support Vector Machines, ECML, 1998.
- [8] A. McCallum and K. Nigam, A Comparison of Event Models for Naive Bayes Text Classification, AAAI, 1998.
- [9] D. Jurafsky and J. H. Martin, Speech and Language Processing, 2nd ed., Pearson, 2009.
- [10] F. Sebastiani, Machine Learning in Automated Text Categorization, ACM Comp. Surveys, 34(1), 2002.
- [11] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016.
- [12] A. Vaswani et al., Attention Is All You Need, NeurIPS, 2017.
- [13] J. Devlin et al., BERT: Pre-training of Deep Bidirectional Transformers, NAACL, 2019.
- [16] Scikit-learn: Machine Learning in Python, <https://scikit-learn.org>
- [17] Kaggle Resume Dataset, <https://www.kaggle.com>
- [18] NLTK Documentation, <https://www.nltk.org>
- [19] SpaCy NLP, <https://spacy.io>